

Exception handling: try-catch, custom exceptions

Part 1: Exception Overview

An Exception is an event that occurs during the execution of a program that disrupts the normal flow of instructions. It is an object that wraps an error event that occurred within a method and contains information about the error, including its type and the state of the program when it happened.

Lets have an example without exception.

Class 1: The Logic/Engine Class

```
// DivisionEngine.java
public class DivisionEngine {

    public int divide(int a, int b) {
        return a / b;
    }

    public static void main(String[] args) {

        DivisionEngine engine = new DivisionEngine();

        System.out.println("--- Starting Division Engine Test ---");

        int successfulResult = engine.divide(10, 2);
        System.out.println("Test A (10 / 2) Result: " + successfulResult);

        System.out.println("Test B (10 / 0) starting... (This will crash the application!)");

        int crashedResult = engine.divide(10, 0);

        System.out.println("Test B Result: " + crashedResult);

    }
}
```

Class 2: The GUI Presentation Class

This class handles the Swing user interface layout and references the `DivisionEngine` class to process the inputs when the button is clicked.

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class DivideByZeroUnprotected extends JFrame {
    private JTextField inputField1, inputField2;
    private JButton calculateButton;
    private JLabel resultLabel;

    private DivisionEngine engine;

    public DivideByZeroUnprotected() {
        engine = new DivisionEngine();

        setTitle("Example 11.2: Unprotected Separation");
        setSize(400, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new GridLayout(4, 1, 10, 10));

        inputField1 = new JTextField("10");
        inputField2 = new JTextField("0");
        calculateButton = new JButton("Divide A by B");
        resultLabel = new JLabel("Result", SwingConstants.CENTER);

        add(inputField1);
        add(inputField2);
        add(calculateButton);
        add(resultLabel);

        calculateButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                executeDivision();
            }
        });
    }

    private void executeDivision() {
        String textA = inputField1.getText().trim();
        String textB = inputField2.getText().trim();
        int numA = Integer.parseInt(textA);
        int numB = Integer.parseInt(textB);
        int result = engine.divide(numA, numB);

        resultLabel.setText("Result: " + result);
    }
}
```

```
// main method
public static void main(String[] args) {
    DivideByZeroUnprotected window = new DivideByZeroUnprotected();
    window.setVisible(true);
}
}
```

When we compile and run this class directly, the console output will look like this:

```
--- Starting Division Engine Test ---
Test A (10 / 2) Result: 5
Test B (10 / 0) starting... (This will crash the application!)
Exception in thread "main" java.lang.ArithmeticException: / by zero
at DivisionEngine.divide(DivisionEngine.java:6)
at DivisionEngine.main(DivisionEngine.java:23)
```

Part 2: The Exception Hierarchy

In Java, all exception and error types are subclasses of the `Throwable` class.

- **Error:** Indicates serious problems that a reasonable application should not try to catch (e.g., `OutOfMemoryError`, `StackOverflowError`). These are system-level failures.
- **Exception:** Indicates conditions that a reasonable application might want to catch.
 - **Checked Exceptions:** Exceptions that are checked at compile-time (e.g., `IOException`, `SQLException`). The compiler forces you to handle them.
 - **Unchecked Exceptions (RuntimeException):** Exceptions that occur at runtime (e.g., `NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`). They usually result from programming logic errors.

Part 3: The try-catch Mechanism

To handle exceptions at runtime, we enclose the risky code within a `try` block and the error-handling logic within one or more `catch` blocks.

```
try {  
    // Code that might throw an exception  
} catch (SpecificExceptionType e) {  
    // Code to handle the exception  
} finally {  
    // Code that ALWAYS executes (cleanup code)  
}
```

By adding a `try-catch` block inside the `main` method, we can intercept the `ArithmeticException` before it crashes the application. This allows us to handle the error gracefully by printing a helpful message to the console instead of letting the program terminate abruptly.

The Protected `DivisionEngine.java` Code

```
// DivisionEngine.java  
public class DivisionEngine {  
  
    // This method remains focused on the math operation  
    public int divide(int a, int b) {  
        return a / b;  
    }  
}
```

```

// The main method now safely catches runtime failures
public static void main(String[] args) {
    DivisionEngine engine = new DivisionEngine();

    System.out.println("-Starting Protected Division Engine Test--");

    int successfulResult = engine.divide(10, 2);
    System.out.println("Test 1 (10 / 2) Result: " + successfulResult);
    System.out.println("-----");

    try {
        System.out.println("Attempting Test 2 (10 / 0)...");

        int result = engine.divide(10, 0);

        System.out.println("Result is " + result);
    } catch (ArithmeticException ex) {
        System.out.println("\n[ERROR INTERCEPTED]");
        System.out.println("Message from System: " + ex.getMessage());
        System.out.println("Resolution: You cannot divide an integer by
zero. Please change your inputs.");
    } finally {
        System.out.println("\nExecution sweep completed. The program
survived the error!");
    }

    System.out.println("\n--End of Program:--");
}
}

```

Multiple Catch Blocks

When a `try` block contains code that can throw more than one type of exception, you can handle each scenario differently by stacking multiple catch blocks. A single `try` block can have multiple `catch` blocks to handle different types of exceptions uniquely.

Always catch the most specific exception first and the most general one last. If you catch `Exception` first, the subsequent catch blocks will be unreachable, causing a compiler error.

Code Example:

In this example, we simulate a common application scenario: loading a user profile where we might encounter an empty input (`NullPointerException`), bad number formatting (`NumberFormatException`), or a mathematical breakdown like dividing by zero (`ArithmeticException`).

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class MultiCatchDemo extends JFrame {
    private JTextField inputField1;
    private JTextField inputField2;
    private JButton calculateButton;
    private JLabel resultLabel;

    public MultiCatchDemo() {
        // Setup simple Swing Layout
        setTitle("Multiple Catch Lab Demonstration");
        setSize(450, 250);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new GridLayout(5, 1, 10, 10));

        inputField1 = new JTextField("Enter Number A");
        inputField2 = new JTextField("Enter Number B");
        calculateButton = new JButton("Divide A by B");
        resultLabel = new JLabel("Result: ", SwingConstants.CENTER);

        add(inputField1);
        add(inputField2);
        add(calculateButton);
        add(resultLabel);

        calculateButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                executeDivision();
            }
        });
    }
}
```

```

private void executeDivision() {
    try {

        String textA = inputField1.getText().trim();
        String textB = inputField2.getText().trim();

        // Scenario 1: Simulating a forced NullPointerException
        if (textA.isEmpty() || textB.isEmpty()) {
            throw new NullPointerException("Input strings are missing.");
        }

        // Scenario 2: NumberFormatException if user enters "abc"
        int numA = Integer.parseInt(textA);
        int numB = Integer.parseInt(textB);

        // Scenario 3: Potential ArithmeticException if numB is 0
        int result = numA / numB;

        // Output
        resultLabel.setForeground(new Color(0, 100, 0));
        resultLabel.setText("Success! Result: " + result);

    }

    // CATCH BLOCK 1: Specific mathematical errors
    catch (ArithmeticException ex) {
        resultLabel.setForeground(Color.RED);
        resultLabel.setText("Math Error: Cannot divide by zero!");
        JOptionPane.showMessageDialog(this, "You cannot divide a number
by zero.", "Arithmetic Error", JOptionPane.ERROR_MESSAGE);
    }

    // CATCH BLOCK 2: Specific data type formatting errors
    catch (NumberFormatException ex) {
        resultLabel.setForeground(Color.RED);
        resultLabel.setText("Format Error: Non-numeric data entered.");
        JOptionPane.showMessageDialog(this, "Please ensure both fields
contain valid integers.", "Input Format Error", JOptionPane.ERROR_MESSAGE);
    }

    // CATCH BLOCK 3: Specific missing value errors
    catch (NullPointerException ex) {
        resultLabel.setForeground(Color.RED);
        resultLabel.setText("Null Error: Fields cannot be blank.");
    }

    // CATCH BLOCK 4: The Generic Global Safety Net (Must be LAST)
    catch (Exception ex) {
        resultLabel.setForeground(Color.RED);
        resultLabel.setText("Unexpected Critical Error occurred.");
        System.out.println("Logged system: " + ex.getMessage());
    }
    finally {
        System.out.println("Try-Catch block execution ended.");
    }
}

```

```
// main method
public static void main(String[] args) {
    MultiCatchDemo window = new MultiCatchDemo();
    window.setVisible(true);
}
}
```

Part 4: Creating Custom Exceptions

While Java provides a rich set of built-in exceptions, business logic often requires domain-specific errors. For instance, in a banking app, throwing an `IllegalArgumentException` for an overdraft is vague. Creating an `InsufficientFundsException` is much more meaningful.

Rules for Custom Exceptions

1. Extend `Exception` if you want a **checked** exception (forced handling).
2. Extend `RuntimeException` if you want an **unchecked** exception (optional handling).
3. Provide a constructor that accepts a custom error message string.

Example Construction:

```
// Custom Checked Exception
public class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message); // Passes message to the parent Exception class
    }
}
```

Complete Code Implementation

```
public class InvalidAgeException extends Exception {

    // Constructor that accepts a custom error message string
    public InvalidAgeException(String message) {
        super(message); // Passes message to the parent Exception class
    }

    public static void checkAgeLimit(int age) throws InvalidAgeException {
        if (age < 18) {
            // Manually creating and throwing our custom exception object
            throw new InvalidAgeException("Age " + age + " is too young! Must be 18 or older.");
        }
        System.out.println("Age verification passed: " + age);
    }
}
```

```

public static void main(String[] args) {
    System.out.println("--- Starting Custom Exception Test --- \n");

    // Test Case 1: Valid Data
    try {
        System.out.println("Executing Test 1 (Age: 25)...");
        checkAgeLimit(25);
        System.out.println("Test 1 completed successfully!\n");
    } catch (InvalidAgeException e) {
        System.out.println("Exception in Test 1: " + e.getMessage());
    }

    System.out.println("-----\n");

    // Test Case 2: Invalid Data (Triggers Exception)
    try {
        System.out.println("Executing Test 2 (Age: 15)...");

        // This line will instantly jump down to the catch block
        checkAgeLimit(15);

        System.out.println("This line will be skipped completely.");
    } catch (InvalidAgeException e) {

        System.out.println("[EXCEPTION CAUGHT] -> " + e.getMessage());
    }

    System.out.println("\n--- End of Program---");
}
}

```

Console Output

When you run this file directly in your IDE, the output will look like this:

```

--- Starting Custom Exception Test ---

Executing Test 1 (Age: 25)...
Age verification passed: 25
Test 1 completed successfully!

-----

Executing Test 2 (Age: 15)...
[EXCEPTION CAUGHT] -> Age 15 is too young! Must be 18 or older.

--- End of Program: The system survived safely ---

```